HASTS 416 Tutorial - Robert Chonge

Code

Robert Chonge (R215903Z)

2026-03-31

- Define the Transition Probability Matrix
- (a) Markov Chain Classification
- (b) Simulate Three Trajectories
- (c) Steady-State Probabilities
- (d) Convergence to Steady-State
- Transition Matrix Heatmap
- Define the Transition Probability Matrix (7 × 7)
- (a) Plot the Markov Chain Diagram
- (b) Classification of States
- Transition Matrix Heatmap
- Question A3
    - Time-Inhomogeneous Markov Chain (Traffic Conditions)
    - Define State Space and Transition Matrices
    - Transition Matrices
    - (a) Theoretical Distribution at 6PM
    - (b) Simulation of 10,000 Trajectories
    - Theoretical vs Empirical Comparison
    - Evolution of Probabilities Over Time
    - Final Distribution Comparison
    - Transition Matrix Heatmaps

#Question A1
5-State Markov Chain Analysis

Hide

```
# Load required libraries
library(markovchain)
library(diagram)
library(ggplot2)
```

Hide

```
library(reshape2)
library(expm)
```

Define the Transition Probability Matrix

Hide

```
P <- matrix(c(
  0.1, 0.3, 0.2, 0.2, 0.2,
  0.2, 0.1, 0.3, 0.2, 0.2,
  0.0, 0.0, 0.5, 0.5, 0.0,
  0.0, 0.0, 0.5, 0.4, 0.1,
  0.0, 0.0, 0.0, 0.0, 1.0
), nrow = 5, byrow = TRUE)

rowSums(P)
```

```
[1] 1 1 1 1 1
```

Hide

```
colnames(P) <- 1:5
rownames(P) <- 1:5
P
```

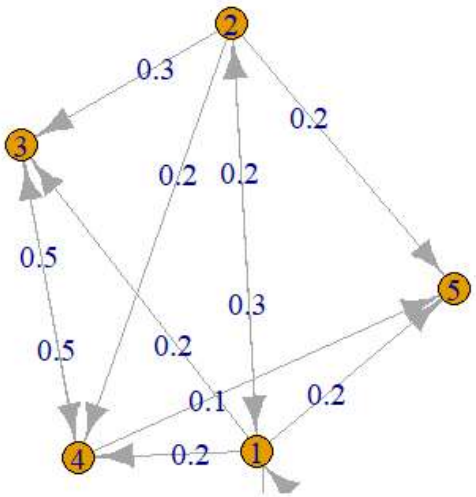
```
    1   2   3   4   5
1 0.1 0.3 0.2 0.2 0.2
2 0.2 0.1 0.3 0.2 0.2
3 0.0 0.0 0.5 0.5 0.0
4 0.0 0.0 0.5 0.4 0.1
5 0.0 0.0 0.0 0.0 1.0
```

(a) Markov Chain Classification

- Plot diagram
- Identify classes
- Determine periods
- Identify absorbing & reflective states

Hide

```
mc <- new("markovchain",
          states = as.character(1:5),
          transitionMatrix = P)

plot(mc)
```



Hide

```
# Summary
summary(mc)
```

```
Unnamed Markov chain  Markov chain that is composed by:
Closed classes:
5
Recurrent classes:
{5}
Transient classes:
{1,2},{3,4}
The Markov chain is not irreducible
The absorbing states are: 5
```

Hide

```
# Communicating classes
classes <- communicatingClasses(mc)
classes
```

```
[[1]]
[1] "1" "2"

[[2]]
[1] "3" "4"

[[3]]
[1] "5"
```

Hide

```
# Class type (recurrent/transient)
class_types <- character(length(classes))

for(i in 1:length(classes)){
  class_states <- classes[[i]]
  is_closed <- TRUE

  for(state in class_states){
    for(next_state in 1:5){
      if(P[as.numeric(state), next_state] > 0 &&
         !(as.character(next_state) %in% class_states)){
        is_closed <- FALSE
        break
      }
    }
  }

  class_types[i] <- ifelse(is_closed, "RECURRENT", "TRANSIENT")
}

class_types
```

```
[1] "TRANSIENT" "TRANSIENT" "RECURRENT"
```

Hide

```
#Periods
period(mc)
```

```
[1] 0
```

Hide

```
# Absorbing states
absorbing_states <- which(diag(P) == 1)
absorbing_states
```

```
5
5
```

Hide

```
# Reflective states (no self-loop)
reflective_states <- which(diag(P) == 0)
reflective_states
```

```
named integer(0)
```

(b) Simulate Three Trajectories

Hide

```
set.seed(123)

simulate_trajectory <- function(start_state, n_steps = 20){
  traj <- numeric(n_steps + 1)
  traj[1] <- start_state

  for(i in 1:n_steps){
    start_state <- sample(1:5, 1, prob = P[start_state, ])
    traj[i + 1] <- start_state
  }

  traj
}
```

Hide

```
set.seed(456)
start_states <- sample(1:5, 3, replace = TRUE)
start_states
```

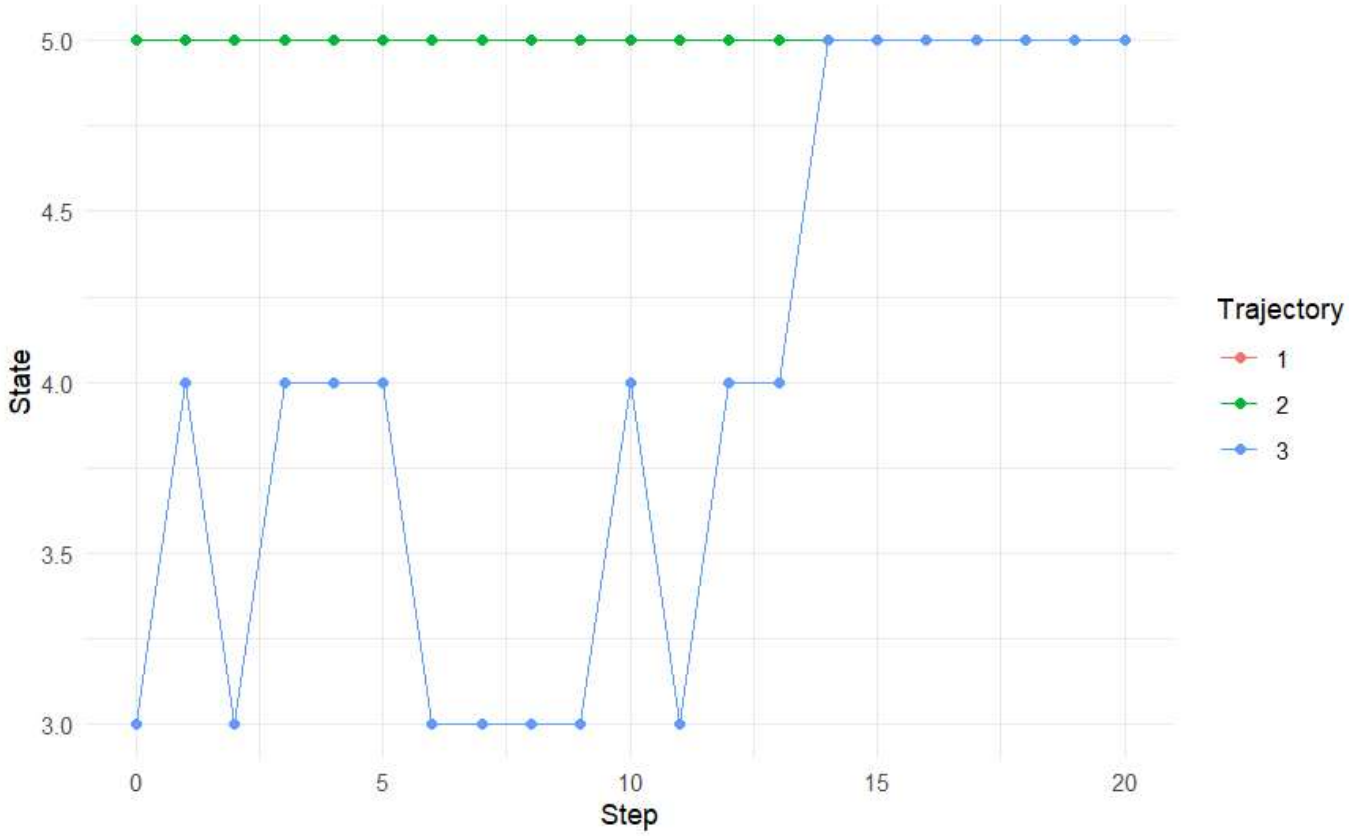
```
[1] 5 5 3
```

Hide

```
trajectories <- lapply(start_states, simulate_trajectory)
```

Hide

```
# Plot trajectories
traj_data <- data.frame(
  Step = rep(0:20, 3),
  State = unlist(trajectories),
  Trajectory = factor(rep(1:3, each = 21))
)

ggplot(traj_data, aes(Step, State, color = Trajectory)) +
  geom_line() +
  geom_point() +
  theme_minimal()
```



(c) Steady-State Probabilities

Hide

```
# Check if chain is irreducible
is_irreducible <- is.irreducible(mc)

# Check if chain is aperiodic (period = 1 for whole chain)
is_aperiodic <- (period(mc) == 1)

# Ergodic if irreducible and aperiodic
is_ergodic <- is_irreducible && is_aperiodic
is_ergodic
```

```
[1] FALSE
```

Hide

```
if(is_ergodic){
  steadyStates(mc)
}
```

Hide

```
# Solve manually
A <- t(P) - diag(5)
A <- rbind(A, rep(1, 5))
b <- c(rep(0, 5), 1)

steady <- solve(t(A) %*% A, t(A) %*% b)
steady / sum(steady)
```

```
  [,1]
1    0
2    0
3    0
4    0
5    1
```

Hide

```
# Solve manually
A <- t(P) - diag(5)
A <- rbind(A, rep(1, 5))
b <- c(rep(0, 5), 1)

steady <- solve(t(A) %*% A, t(A) %*% b)
steady / sum(steady)
```

```
  [,1]
1    0
2    0
3    0
4    0
5    1
```

(d) Convergence to Steady-State

Hide

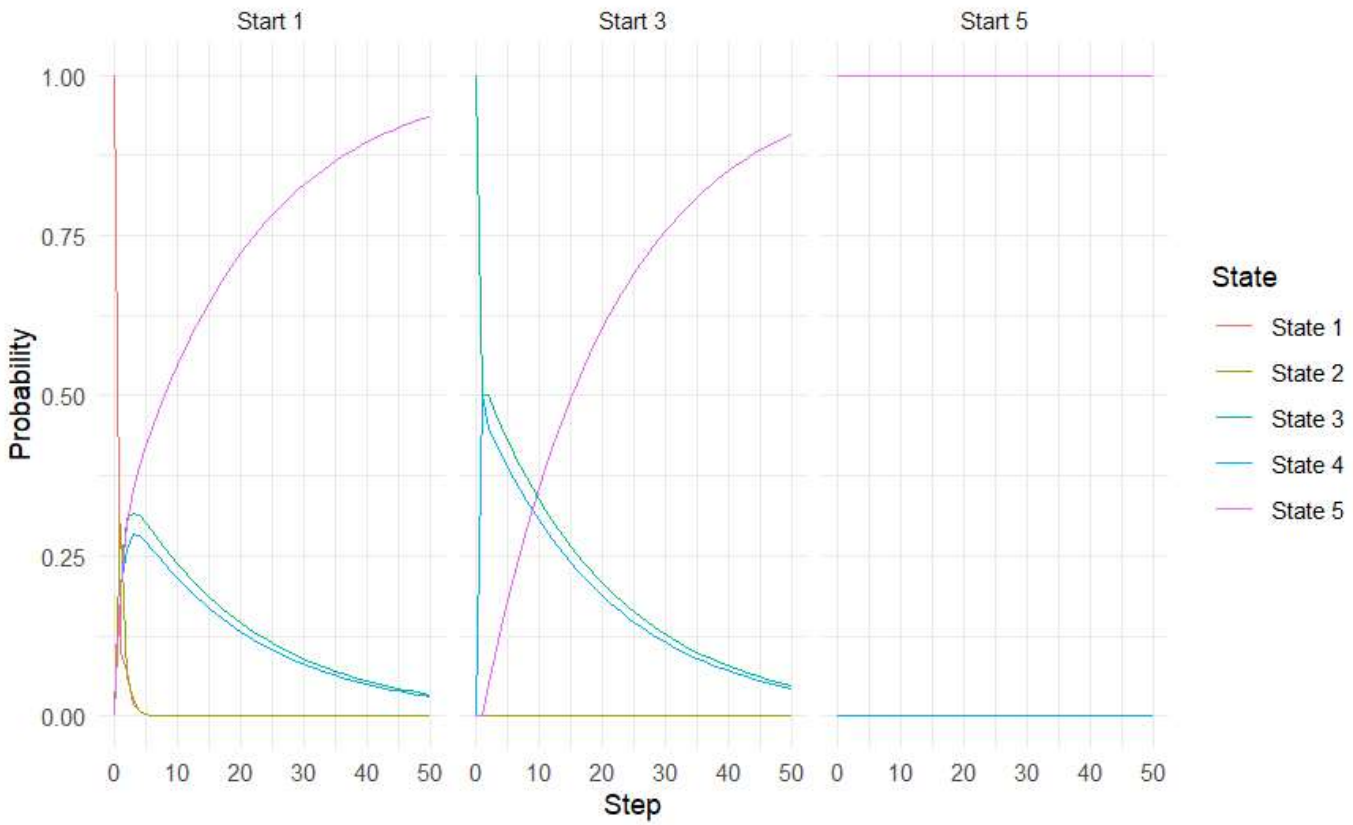
```
n_steps <- 50
start_states_conv <- c(1, 3, 5)

prob_over_time <- array(0, dim = c(3, n_steps + 1, 5))
```

Hide

```
for(i in 1:3){
  dist <- rep(0, 5)
  dist[start_states_conv[i]] <- 1
  prob_over_time[i, 1, ] <- dist

  for(step in 2:(n_steps + 1)){
    dist <- dist %*% P
    prob_over_time[i, step, ] <- dist
  }
}
```

Hide

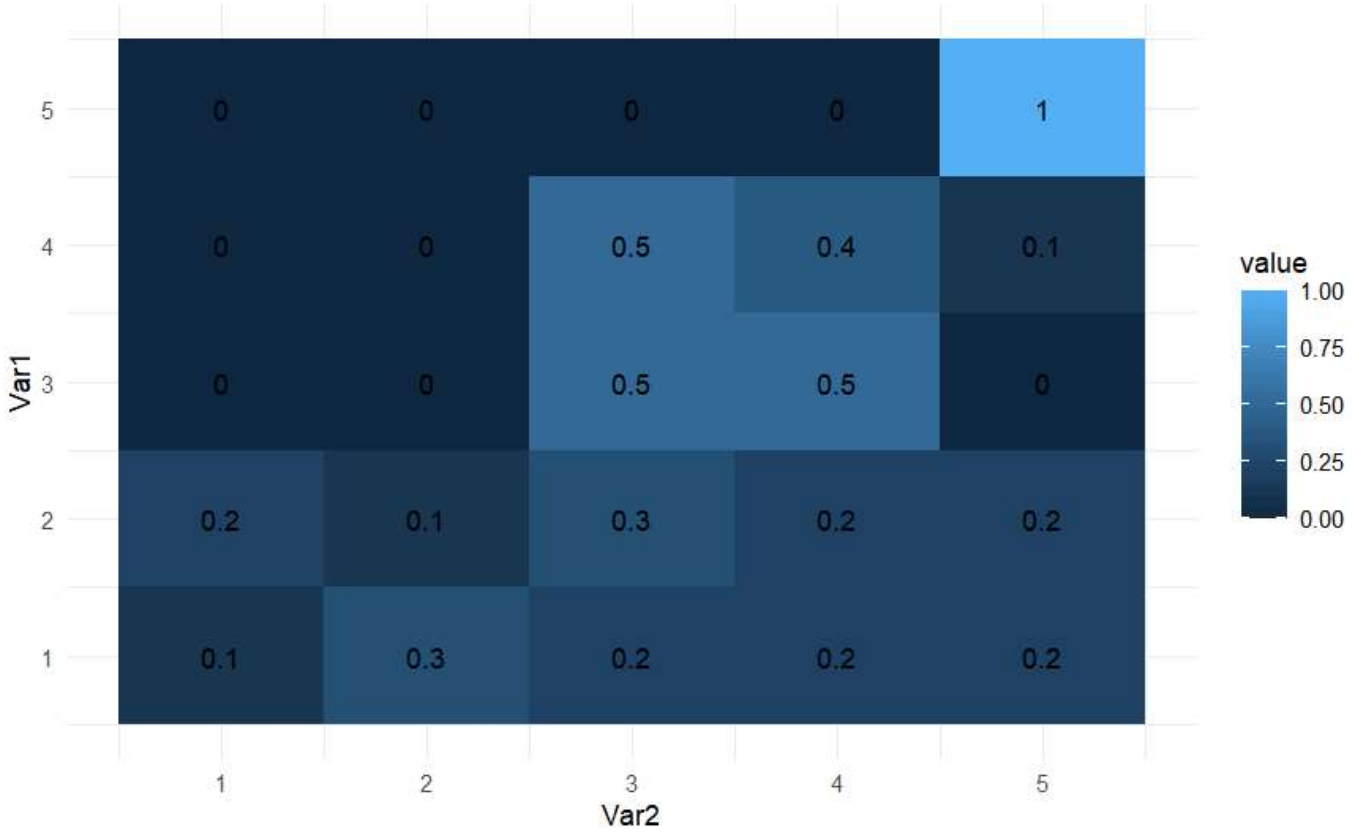
```
# Plot convergence
plot_data <- data.frame()

for(i in 1:3){
  for(s in 1:5){
    plot_data <- rbind(plot_data, data.frame(
      Step = 0:n_steps,
      Probability = prob_over_time[i,,s],
      Start = paste("Start", start_states_conv[i]),
      State = paste("State", s)
    ))
  }
}

ggplot(plot_data, aes(Step, Probability, color = State)) +
  geom_line() +
  facet_wrap(~Start) +
  theme_minimal()
```



Transition Matrix Heatmap

Hide

```
P_melted <- melt(P)

ggplot(P_melted, aes(Var2, Var1, fill = value)) +
  geom_tile() +
  geom_text(aes(label = round(value,2))) +
  theme_minimal()
```



#Question A2
7-State Markov Chain Analysis

Define the Transition Probability Matrix (7 × 7)

Hide

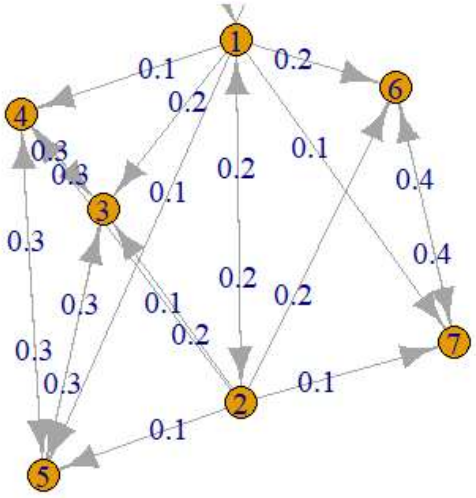
```
P <- matrix(c(
  0.1, 0.2, 0.2, 0.1, 0.1, 0.2, 0.1,
  0.2, 0.1, 0.2, 0.1, 0.1, 0.2, 0.1,
  0.0, 0.0, 0.4, 0.3, 0.3, 0.0, 0.0,
  0.0, 0.0, 0.3, 0.4, 0.3, 0.0, 0.0,
  0.0, 0.0, 0.3, 0.3, 0.4, 0.0, 0.0,
  0.0, 0.0, 0.0, 0.0, 0.0, 0.6, 0.4,
  0.0, 0.0, 0.0, 0.0, 0.0, 0.4, 0.6
), nrow = 7, byrow = TRUE)

round(rowSums(P), 2)
```

```
[1] 1 1 1 1 1 1 1
```

(a) Plot the Markov Chain Diagram

Hide

```
mc <- new("markovchain",
          states = as.character(1:7),
          transitionMatrix = P)

plot(mc)
```



Hide

```
# Save diagram
#dev.copy(png, "A2_diagram.png", width = 800, height = 600)
#dev.off()
```

(b) Classification of States

#Communicating Classes

Hide

```
classes <- communicatingClasses(mc)
classes
```

```
[[1]]
[1] "1" "2"

[[2]]
[1] "3" "4" "5"

[[3]]
[1] "6" "7"
```

#Transient Vs Recurrent

Hide

```
class_types <- character(length(classes))

for(i in 1:length(classes)){
  class_states <- classes[[i]]
  is_closed <- TRUE

  for(state in class_states){
    s_idx <- as.numeric(state)
    for(next_s in 1:7){
      if(P[s_idx, next_s] > 0 &&
         !(as.character(next_s) %in% class_states)){
        is_closed <- FALSE
        break
      }
    }
  }

  class_types[i] <- ifelse(is_closed, "RECURRENT", "TRANSIENT")
}

class_types
```

```
[1] "TRANSIENT" "RECURRENT" "RECURRENT"
```

#Period

Hide

```
period(mc)
```

```
[1] 0
```

#Absorbing states

Hide

```
absorbing <- which(diag(P) == 1)
absorbing
```

```
integer(0)
```

#Reflecting states

Hide

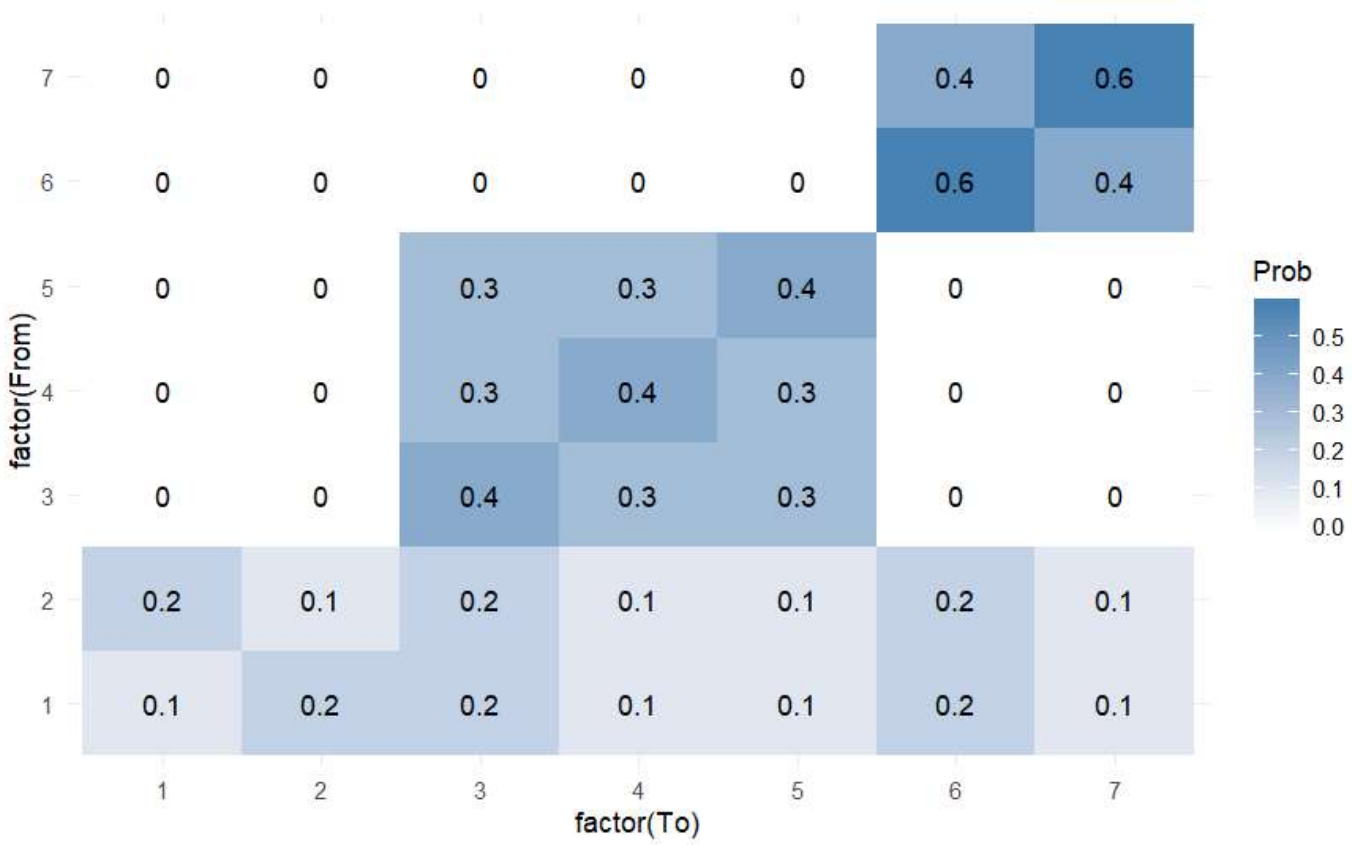
```
reflecting <- which(diag(P) == 0)
reflecting
```

```
integer(0)
```

Transition Matrix Heatmap

Hide

```
P_melt <- melt(P)
colnames(P_melt) <- c("From", "To", "Prob")

ggplot(P_melt, aes(x = factor(To), y = factor(From), fill = Prob)) +
  geom_tile() +
  geom_text(aes(label = round(Prob, 2))) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  theme_minimal()
```



Hide

```
ggsave("A2_heatmap.png", width = 8, height = 6)
```

Question A3

Time-Inhomogeneous Markov Chain (Traffic Conditions)

Define State Space and Transition Matrices

Hide

```
states <- c("Light", "Heavy", "Jammed")

P1 <- matrix(c(
  0.4, 0.4, 0.2,
  0.3, 0.4, 0.3,
  0.0, 0.1, 0.9
), nrow = 3, byrow = TRUE)

P2 <- matrix(c(
  0.1, 0.5, 0.4,
  0.1, 0.3, 0.6,
  0.0, 0.1, 0.9
), nrow = 3, byrow = TRUE)

round(rowSums(P1), 2)
```

```
[1] 1 1 1
```

Hide

```
round(rowSums(P2), 2)
```

```
[1] 1 1 1
```

Transition Matrices

Hide

```
colnames(P1) <- states; rownames(P1) <- states
colnames(P2) <- states; rownames(P2) <- states

P1
```

```
       Light Heavy Jammed
Light    0.4   0.4    0.2
Heavy    0.3   0.4    0.3
Jammed   0.0   0.1    0.9
```

Hide

```
P2
```

```
       Light Heavy Jammed
Light    0.1   0.5    0.4
Heavy    0.1   0.3    0.6
Jammed   0.0   0.1    0.9
```

(a) Theoretical Distribution at 6PM

Hide

```
initial <- c(1, 0, 0)

# 4PM
dist_4PM <- initial
for(i in 1:9){
  dist_4PM <- dist_4PM %*% P1
}
dist_4PM
```

```
         Light     Heavy    Jammed
[1,] 0.1005492 0.1899846 0.7094662
```

Hide

```
# 6PM
dist_6PM <- dist_4PM
for(i in 1:6){
  dist_6PM <- dist_6PM %*% P2
}
dist_6PM
```

```
          Light    Heavy    Jammed
[1,] 0.01480077 0.132596 0.8526032
```

Hide

```
# Verification
dist_6PM_alt <- initial %*% (P1 %^% 9 %*% P2 %^% 6)
dist_6PM_alt
```

```
          Light     Heavy    Jammed
[1,] 0.02710305 0.1722042 0.8006928
```

(b) Simulation of 10,000 Trajectories

Hide

```
set.seed(123)

simulate <- function(){
  state <- 1

  for(i in 1:9){
    state <- sample(1:3, 1, prob = P1[state, ])
  }

  for(i in 1:6){
    state <- sample(1:3, 1, prob = P2[state, ])
  }

  state
}
```

Hide

```
n_sim <- 10000
results <- replicate(n_sim, simulate())

empirical <- table(results) / n_sim
names(empirical) <- states

empirical
```

```
 Light  Heavy Jammed
0.0146 0.1287 0.8567
```

Theoretical vs Empirical Comparison

Hide

```
comparison <- data.frame(
  State = states,
  Theoretical = dist_6PM,
  Empirical = as.numeric(empirical),
  Difference = dist_6PM - as.numeric(empirical)
)

comparison
```

Hide

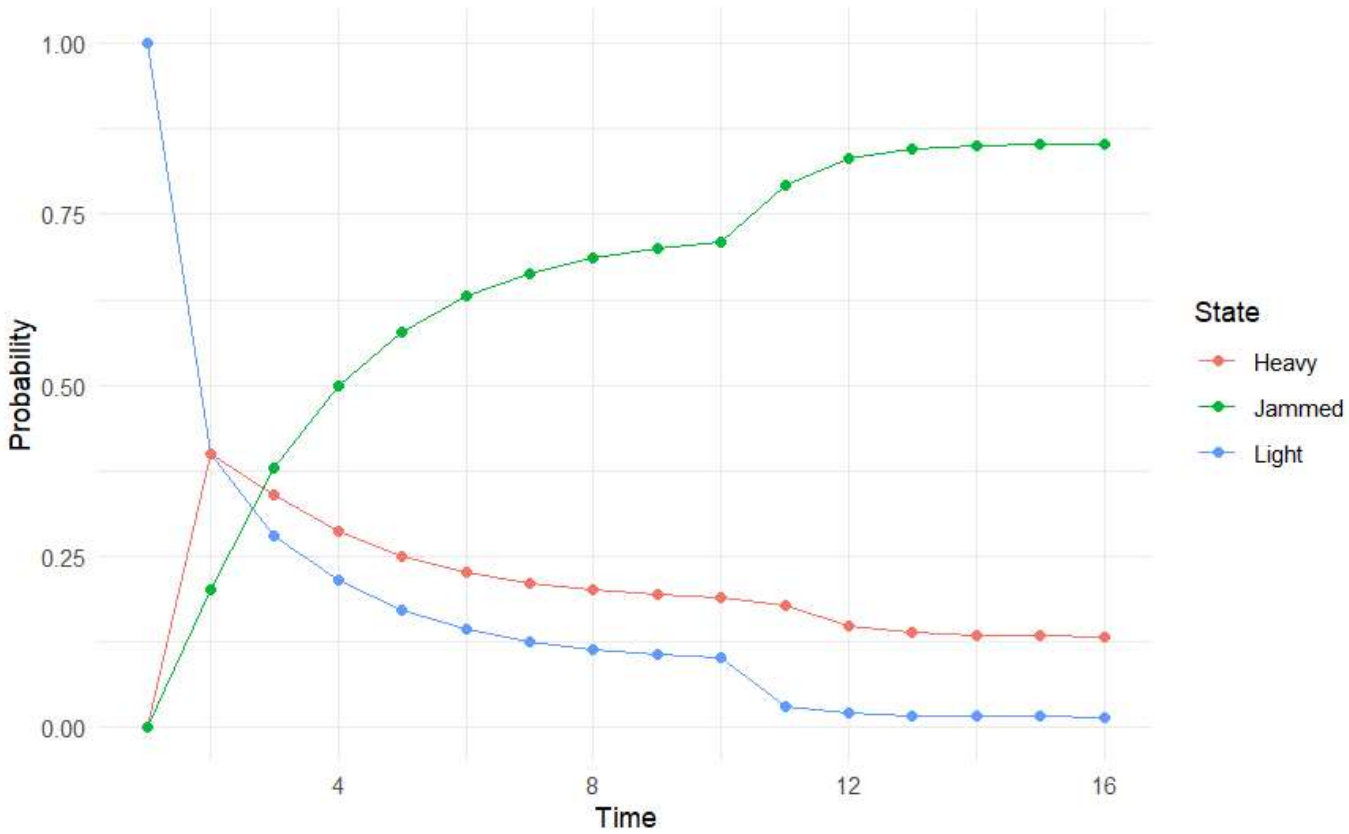
```
chisq.test(x = as.numeric(empirical) * n_sim, p = dist_6PM)
```

```


    Chi-squared test for given probabilities

data:  as.numeric(empirical) * n_sim
X-squared = 1.3688, df = 2, p-value = 0.5044
```

Evolution of Probabilities Over Time

Hide

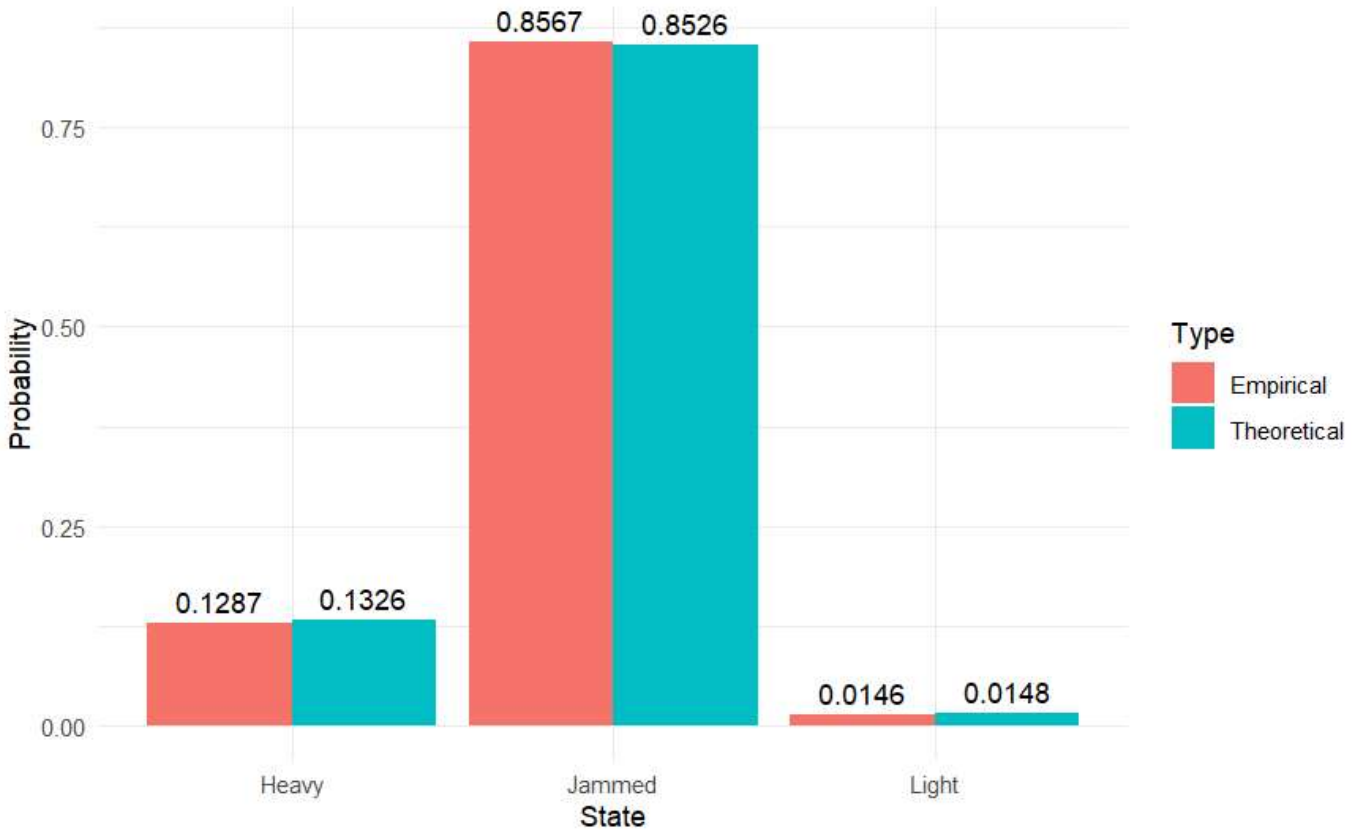
```
time_labels <- c("1PM","1:20","1:40","2PM","2:20","2:40",
                 "3PM","3:20","3:40","4PM","4:20","4:40",
                 "5PM","5:20","5:40","6PM")

dist_time <- matrix(0, nrow = 16, ncol = 3)
dist_time[1, ] <- initial

for(t in 2:10){
  dist_time[t, ] <- dist_time[t-1, ] %*% P1
}
for(t in 11:16){
  dist_time[t, ] <- dist_time[t-1, ] %*% P2
}

plot_data <- data.frame(
  Time = rep(1:16, 3),
  State = rep(states, each = 16),
  Probability = as.vector(dist_time)
)

ggplot(plot_data, aes(Time, Probability, color = State)) +
  geom_line() +
  geom_point() +
  theme_minimal()
```



Final Distribution Comparison

Hide

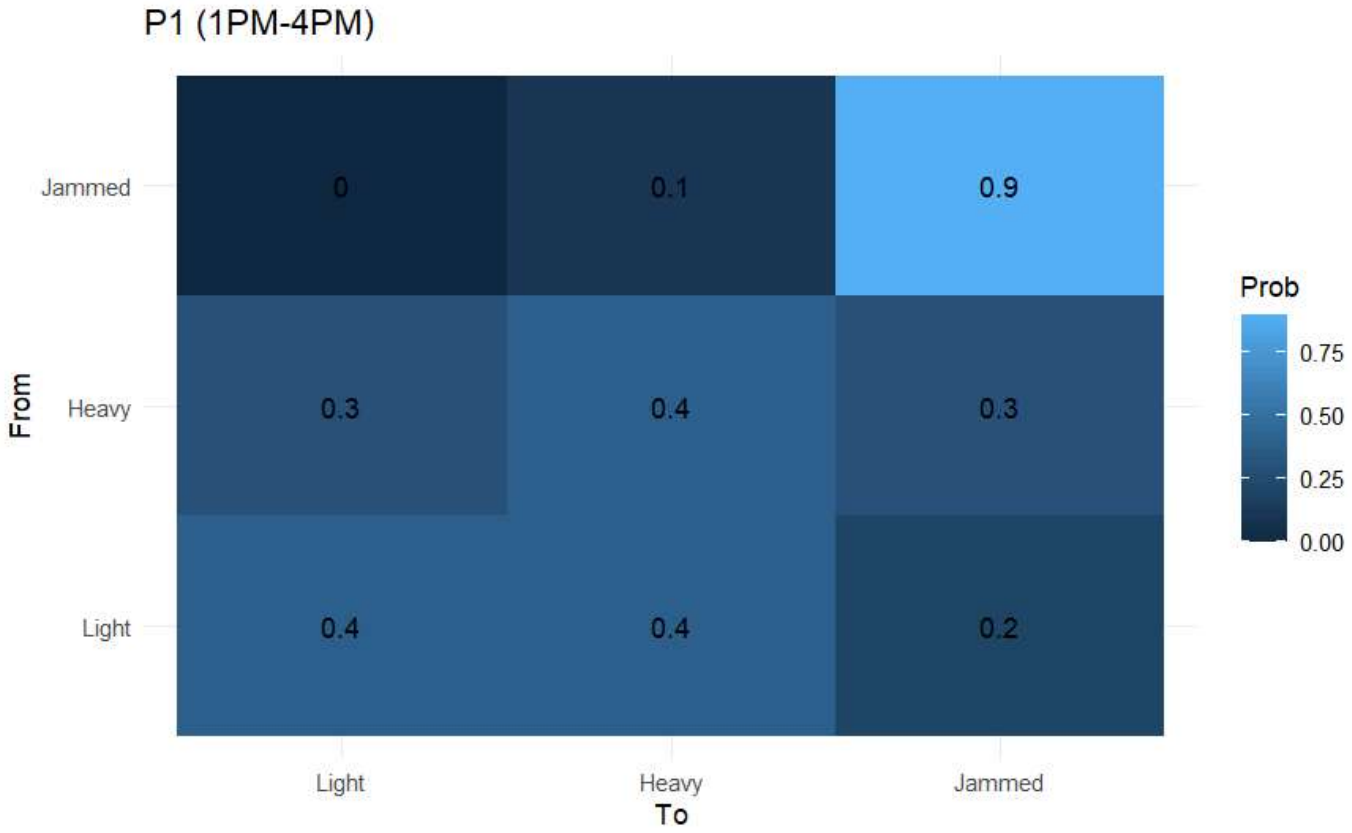
```
comp_data <- data.frame(
  State = rep(states, 2),
  Type = rep(c("Theoretical", "Empirical"), each = 3),
  Probability = c(dist_6PM, as.numeric(empirical))
)

ggplot(comp_data, aes(State, Probability, fill = Type)) +
  geom_bar(stat = "identity", position = "dodge") +
  geom_text(aes(label = round(Probability, 4)),
            position = position_dodge(0.9), vjust = -0.5) +
  theme_minimal()
```



Transition Matrix Heatmaps

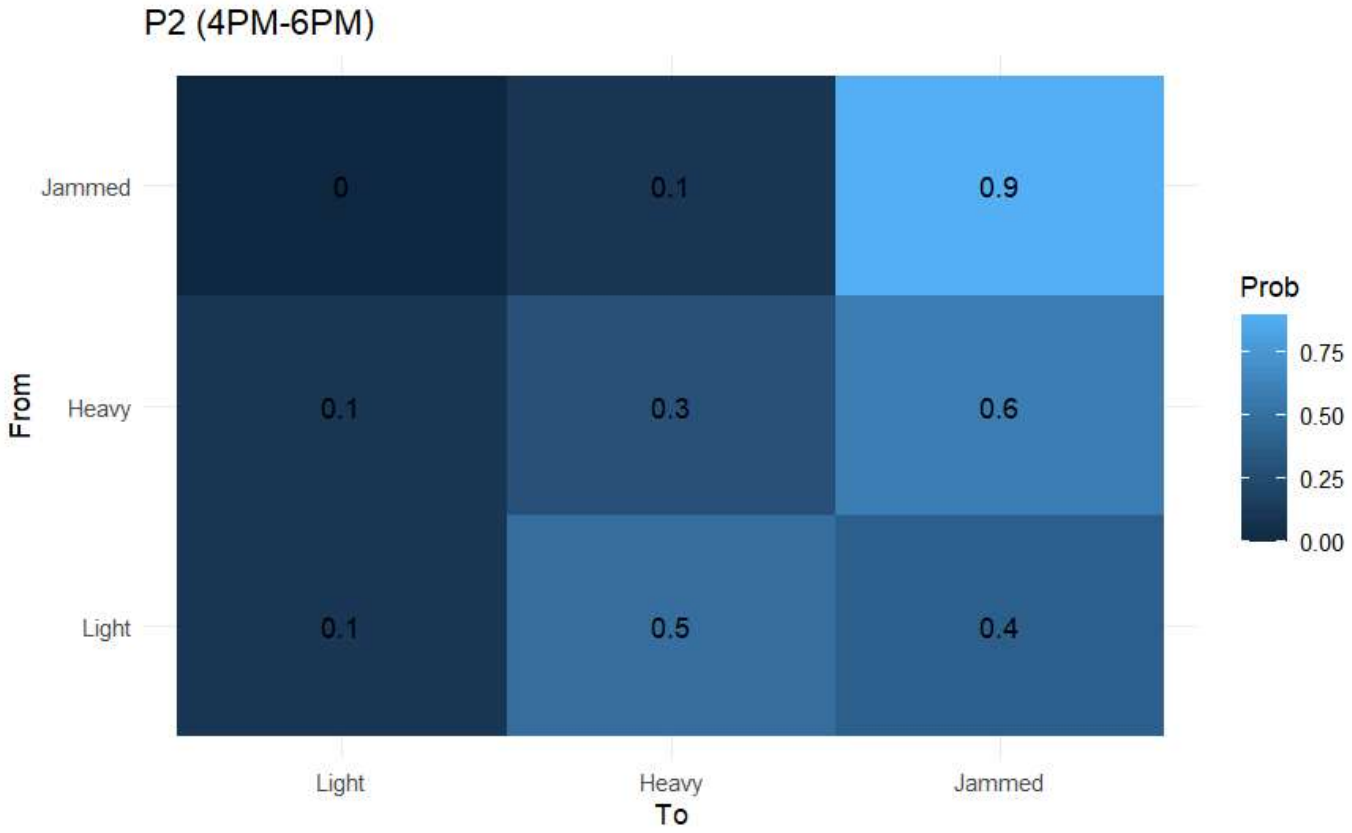
Hide

```
# Heatmap for P1
P1_melt <- melt(P1)
colnames(P1_melt) <- c("From","To","Prob")

ggplot(P1_melt, aes(To, From, fill = Prob)) +
  geom_tile() +
  geom_text(aes(label = round(Prob,2))) +
  ggtitle("P1 (1PM-4PM)") +
  theme_minimal()
```



Hide

```
# Heatmap for P2
P2_melt <- melt(P2)
colnames(P2_melt) <- c("From","To","Prob")

ggplot(P2_melt, aes(To, From, fill = Prob)) +
  geom_tile() +
  geom_text(aes(label = round(Prob,2))) +
  ggtitle("P2 (4PM-6PM)") +
  theme_minimal()
```


P2 (4PM-6PM)
Jammed
Heavy
Light
From
0
0.1
0.9
0.1
0.3
0.6
0.1
0.5
0.4
Light
Heavy
Jammed
To
Prob
0.75
0.50
0.25
0.00